

1. Why is it recommended to explicitly assign values to enum members in some cases?

Because it makes the values clear and predictable, especially when the enum values are stored in a database or used in communication with other systems.

2. What happens if you assign a value to an enum member that exceeds the underlying type's range?

You get a **compile-time error** because the value cannot fit in the underlying type.

3. What is the purpose of the `virtual` keyword when used with properties?

It allows a derived class to **override the property** and provide its own implementation.

4. Why can't you override a sealed property or method?

Because `sealed` means the member **cannot be overridden anymore** in any further derived class.

5. What is the key difference between static and object members?

- **Static member:** belongs to the class itself and is shared by all objects.
- **Object (instance) member:** belongs to a specific object, so each object has its own copy.

6. Can you overload all operators in C#? Explain why or why not.

No. C# only allows overloading **certain operators**. Some operators, such as `=`, `..`, `?:`, and `??`, cannot be overloaded because their behavior is fixed by the language.

7. When should you consider changing the underlying type of an enum?

When you have many enum values and want to **reduce memory usage**, or when you need the enum to match a specific data type used by another system.

8. Why can't a static class have instance constructors?

Because a static class **cannot have objects instantiated from it**, so an instance constructor would have no purpose.

9. What are the advantages of using `Enum.TryParse` over direct parsing with `int.Parse`?

`Enum.TryParse` doesn't throw an exception when the input is invalid. It simply returns `false`, making the program **safer and easier to handle**.

10. What is the difference between overriding `Equals` and `==` for object comparison in C# `struct` and `class`?

`Equals` can be overridden to define **logical/value-based equality**.

`==` depends on the type:

- For **classes**, it normally compares references unless overloaded.
- For **structs**, `==` isn't automatically available unless the operator is explicitly overloaded.

11. Why is overriding ToString beneficial when working with custom classes?

It allows you to define a **readable representation** of the object instead of getting the default class name when printing it.

12. Can generics be constrained to specific types in C#? Provide an example.

Yes. You can use constraints such as where T : class, struct, or a specific base class/interface.

Example:

```
static T GetItem<T>(T item) where T : class
{
    return item;
}
```

This means T must be a reference type.

13. What are the key differences between generic methods and generic classes?

- **Generic method:** only the method uses a type parameter.
- **Generic class:** the whole class can use the type parameter in its properties, fields, and methods.

14. Why might using a generic swap method be preferable to implementing custom methods for each type?

Because one generic method can work with **many different types**, so you avoid writing separate swap methods for each type.

15. How can overriding Equals for the Department class improve the accuracy of searches?

It allows departments to be compared based on their **actual values**, such as department ID or name, instead of comparing whether they are the exact same object.

16. Why is == not implemented by default for structs?

Because C# doesn't automatically know **which fields or properties should determine equality** for a custom struct. The programmer can define == when a specific comparison behavior is needed.

Part 2

2. What do we mean by Generalization concept using Generics?

Generalization using Generics means writing code that can work with **different data types without rewriting the same code** for each type.

3. What do we mean by Hierarchy Design in Real Business?

Hierarchy Design means organizing entities or roles in **levels based on their relationships and responsibilities**.